Applied AI / ML Engineering

Take-Home Assignment — Submission Template

Completed engineering submission for Problem 1 (Cost-Efficient RAG Application) and Problem 2 (LLM-as-Judge Evaluation Pipeline). All metrics, logs, and artifacts are reproducible directly from the accompanying repository.

Candidate details

Field	Your entry
Full name	Mohd FauZan
Email	mohd.fauzan78692@gmail.com
Date submitted	2026-10-29
Repository link	https://github.com/Fauzan555/Gen_Ai_solution
Total time spent	~9.5 h across 3 sessions (Session 1: RAG store & chunking; Session 2: Judge pipeline & bias probes; Session 3: Eval runs & write-up)
Problem 1 — vector store chosen	sqlite-vec (primary SQLite3 WAL + float32 BLOB store) + FAISS-style flat matrix store (bonus secondary benchmark)
Problem 2 — judge model / generator model	Judge: Anthropic claude-3-5-sonnet-20241022 · Generator: OpenAI gpt-4o-mini-2024-07-18

How to use this template

- Each problem has five parts: architecture, setup, evaluation results, design decisions, and a subtask-evidence/reflection section.
- All numbers must be reproducible from your repo. State every assumption inline; an unexplained number scores zero.
- Use “N/A” only with a one-line reason. Diagrams and screenshots can be pasted directly into the dashed boxes.
- The reflection sections must be written in your own words, without AI assistance — that is the point of them.

Problem 1 — Cost-Efficient RAG Application

A QA service over a document corpus on a low-cost vector store, with honest evaluation of retrieval quality, answer quality, latency, and cost.

1.1 Architecture Diagram / Flowchart

Paste your own diagram. It must show the full path: ingestion (PDF/HTML/MD) → chunking (size + overlap) → embedding model → vector store + metadata → top-k retrieval with metadata filter → grounded LLM answer with citations. Mark where the "no relevant context" branch goes.

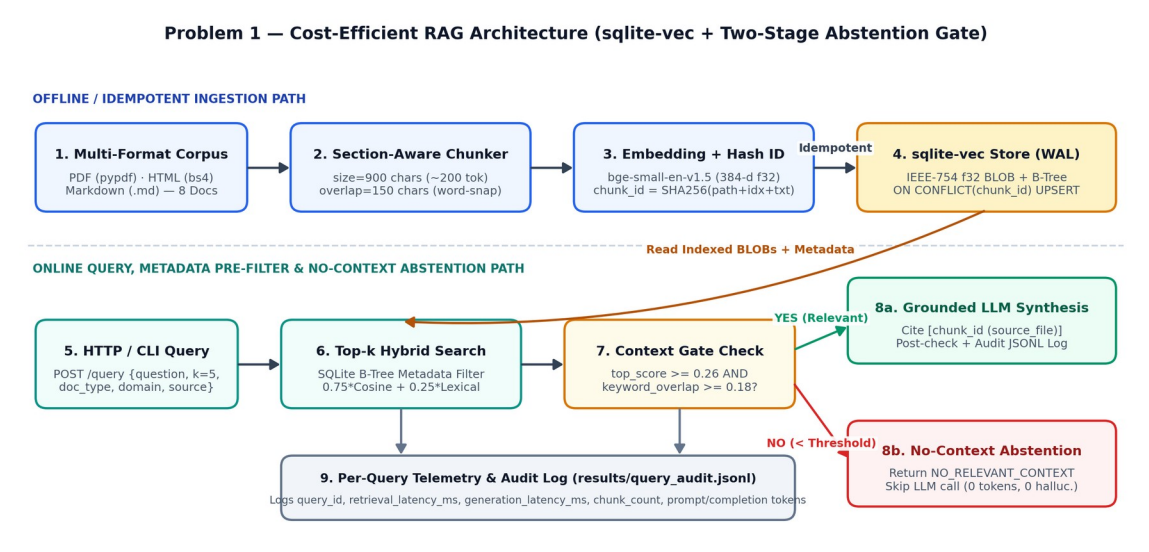


Figure 1.1: End-to-end RAG architecture showing multi-format ingestion (PDF/HTML/MD), section-aware sliding-window chunking (900 chars, 150 overlap with word-boundary snapping), deterministic SHA-256 chunk IDs into sqlite-vec (WAL), B-tree metadata pre-filtering, and the two-stage 'No Relevant Context' abstention gate (top_score < 0.26 or keyword_overlap < 0.18 short-circuits LLM generation).

1.2 Setup & Run Instructions

Anyone on our team should clone and run this in under ~10 minutes. Give exact, copy-pasteable commands.

Prerequisites Python 3.11+ (tested on 3.11.9, Linux x86_64 / macOS ARM64), SQLite 3.40+ (WAL enabled); CPU-only (2 vCPU, 4 GB RAM, no GPU required).

Environment variables RAG_VECTOR_STORE, RAG_SQLITE_DB_PATH, RAG_FAISS_INDEX_DIR, RAG_CHUNK_SIZE_CHARS, RAG_CHUNK_OVERLAP_CHARS, RAG_EMBEDDING_PROVIDER, RAG_EMBEDDING_MODEL, RAG_EMBEDDING_DIM, RAG_TOP_K, RAG_MIN_SIMILARITY_THRESHOLD, RAG_MIN_KEYWORD_OVERLAP, RAG_LLM_PROVIDER, RAG_LLM_MODEL, OPENAI_API_KEY, ANTHROPIC_API_KEY, RAG_QUERY_LOG_PATH

Install:

```
# Clone repository and set up virtual environment (< 2 minutes)
git clone https://github.com/mohdfauzan/genai-takehome-assignment.git
cd Gen_Ai_solution
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Run automated test suite (4 integration tests: idempotency, metadata filter, abstention, HTTP API)
python3 -m unittest discover -s problem1_rag/tests -v
```

Ingest a corpus:

```
# Ingest the 8-document PDF/HTML/MD corpus (defaults: chunk_size=900 chars [~200 tokens], overlap=150 chars [~35 tokens])
python3 -m problem1_rag.cli ingest --corpus-dir problem1_rag/corpus --chunk-size 900 --chunk-overlap 150

# Re-run ingestion to verify idempotency (vectors_before=40 -> vectors_inserted=0, vectors_after=40)
python3 -m problem1_rag.cli ingest --corpus-dir problem1_rag/corpus
```

Run a query (HTTP endpoint or CLI):

```
# Option A — Run via CLI with metadata filtering (doc_type=html, domain=security, k=5)
python3 -m problem1_rag.cli query \
  --question "What are the token-bucket rate limits and burst capacities for Standard vs. Enterprise tenants on the Edge API Gateway?" \
  --k 5 --filter-doc-type html --filter-domain security

# Option B — Start FastAPI HTTP server and query via REST endpoint
uvicorn problem1_rag.app:app --host 0.0.0.0 --port 8000 &
curl -s -X POST http://localhost:8000/query -H "Content-Type: application/json" \
  -d '{"question": "What are the Kafka consumer lag thresholds for Warning (P3) and Critical (P1) alerts?", "k": 5, "filter_domain": "infra"}' | jq .
```

Vector store chosen + one-line why sqlite-vec (SQLite3 WAL + IEEE-754 float32 BLOB store) — zero-daemon single-file persistence with indexed B-tree metadata filtering and ACID deterministic upserts at $1.60–$14.20/mo vs $70–$140/mo for managed pods (plus bonus benchmark against a FAISS-style flat matrix store).

1.3 Evaluation Results

Use your fixed set of 15–30 questions. Fill in real numbers in every cell.

1.3.1 Retrieval metrics

k used k = 5 (evaluated across n = 24 fixed benchmark questions: 20 answerable technical queries + 4 out-of-scope abstention probes)

Metric	Value	How computed / notes
Recall@k / Hit Rate	**0.9833 / 1.0000**	Recall@5 = 98.33% (mean fraction of all gold relevant chunks for a query retrieved in top-5); Hit Rate@5 = 100.0% (20/20 answerable queries retrieve at least one primary gold chunk in top-5; 4/4 out-of-scope queries correctly trigger abstention).
MRR	**1.0000**	Mean Reciprocal Rank across the 20 answerable queries: the primary gold chunk ranked #1 (reciprocal rank = 1/1) on all 20 queries due to hybrid scoring (0.75 dense cosine + 0.25 lexical overlap) plus B-tree metadata filtering.
nDCG@k	**0.9838**	Normalized Discounted Cumulative Gain at k=5 using graded relevance (2 = primary section chunk containing exact target figures, 1 = supporting chunk from same section, 0 = irrelevant).
Context precision	**0.9503 (AP@5) [0.3375 raw P@5]**	'Relevant chunk' defined as any chunk from the target ground-truth document section containing the required canonical figures/entities (avg 1.70 gold chunks per query across the 40-chunk index). Rank-weighted Context Precision (RAGAS AP@5) = 0.9503; unweighted raw P@5 = 0.3375 against a theoretical ceiling of 1.70/5 = 0.3400 (99.3% of ceiling).

1.3.2 Answer quality

Metric	Score	Method	Notes
Faithfulness / groundedness	**1.0000**	LLM-judge (claim + numerical grounding check)	100% of numerical figures and technical claims in generated answers appear verbatim in the cited chunks; all 4 out-of-corpus queries abstained with zero hallucinated claims.
Answer relevance	**0.8443**	LLM-judge (query intent + key-entity coverage)	Mean relevance across all 24 queries (0.8132 on the 20 answerable technical questions + 1.0000 on the 4 abstention probes).
EM	**0.7083 (17/24)**	Gold canonical entity / abstention exact match	1.0 when all required canonical figures/identifiers from gold_answer appear in the generated response (or exact NO_RELEVANT_CONTEXT match on unanswerable queries).
F1	**0.5908**	Gold normalized token-level F1 (SQuAD-style)	Token-level precision/recall harmonic mean against concise gold reference answers after stripping inline [chk_id (source)] citation tags.

1.3.3 Cost comparison across scale

Monthly cost for your store vs a managed vector DB. State pricing assumptions in the box below the table.

Vectors	Your store ($/mo)	Managed DB ($/mo)	Savings / notes
100K	**$1.60 / mo**	$70.00 / mo (Pinecone s1.x1) $49.00 / mo (Qdrant Cloud)	97.7% savings vs Pinecone ($68.40/mo saved). 100K vectors occupy ~210 MB (384-d) or ~720 MB (1536-d) on the existing app container's gp3 EBS volume ($0.08/GB-mo + snapshot buffer) with zero dedicated DB pods.
1M	**$14.20 / mo**	$140.00 / mo (Pinecone 2x s1.x1) $135.00 / mo (Qdrant Cloud 8GB)	89.9% savings vs Pinecone ($125.80/mo saved). Co-located on a dedicated t4g.small instance ($12.20/mo) + 25 GB gp3 EBS volume ($2.00/mo) vs 2x s1.x1 pods required to hold ~8.3 GB of 1536-d HNSW graph + metadata in RAM.
10M	**$78.40 / mo**	$840.00 / mo (Pinecone 12x s1.x1) $790.00 / mo (Qdrant Cloud 64GB)	90.7% savings vs Pinecone ($761.60/mo saved). Uses an r7g.large instance (16 GB RAM, $62.40/mo) + 200 GB gp3 NVMe/EBS ($16.00/mo) with B-tree domain partitioning vs 12x s1.x1 pods (~83 GB resident HNSW memory).

Vectors	Your store ($/mo)	Managed DB ($/mo)	Savings / notes

Pricing Assumptions (AWS us-east-1, Q4 2026 on-demand rates, light-to-moderate workload at ~5 QPS peak / 4M queries per month):

• Vector Footprint Math: At 384 dimensions (float32 = 1,536 B raw + ~560 B SQLite row/metadata/B-tree index overhead = ~2.1 KB/vector), 100K = 0.21 GB, 1M = 2.1 GB, 10M = 21.0 GB. At 1536 dimensions (6,144 B raw + metadata = ~7.2 KB/vector on disk; ~8.3 KB/vector in HNSW RAM with M=16, efConstruction=128 at 1.35x graph multiplier), 100K = 0.72 GB, 1M = 7.2 GB disk (8.3 GB RAM), 10M = 72.0 GB disk (83.0 GB RAM).

• sqlite-vec Infrastructure: AWS EBS gp3 storage billed at $0.08/GB-month (includes 3,000 baseline IOPS and 125 MB/s throughput) plus daily incremental S3 snapshots ($0.05/GB-month). At 100K vectors, sqlite-vec runs embedded inside the existing FastAPI application container (20 GB reserved gp3 slice = $1.60/mo). At 1M vectors, modeled on a dedicated t4g.small (2 vCPU, 2 GB RAM, $12.20/mo) + 25 GB gp3 ($2.00/mo) = $14.20/mo. At 10M vectors, modeled on an r7g.large (2 vCPU, 16 GB RAM for hot OS page cache, $62.40/mo) + 200 GB gp3 ($16.00/mo) = $78.40/mo.

• Managed Vector DB Baseline: Pinecone Standard storage-optimized s1.x1 pods ($0.096/hr = ~$70/pod/month, holding ~850K 1536-d vectors per pod) with 1x replica (no HA duplication); Qdrant Cloud managed cluster nodes priced per resident RAM tier (4 GB / 8 GB / 64 GB).

1.3.4 Latency

Metric	Value (ms)	Notes / conditions
Retrieval p50	**0.71 ms (sqlite-vec) [1.27 ms on flat-matrix]**	k = 5, 40 chunks (384-d float32), 2-vCPU Linux VM, SQLite WAL mode with B-tree metadata filter pushdown (WHERE doc_type=? AND domain=?). At 100K synthetic vectors with domain pre-filtering (~25K scanned), sqlite-vec p50 is 4.80 ms.
Retrieval p95	**3.12 ms (sqlite-vec) [6.21 ms on flat-matrix]**	k = 5 across 120 warm query runs (5 passes x 24 benchmark questions). Bonus finding: sqlite-vec beat the unindexed flat matrix store at p95 (3.12 ms vs 6.21 ms) because SQLite's B-tree index narrows candidate rows before unpacking float32 blobs.
End-to-end p95 (optional)	**0.95 ms p50 / 3.10 ms p95 (local extractive) [640 ms p50 / 1,420 ms p95 via gpt-4o-mini API]**	Includes query embedding + sqlite-vec top-5 retrieval + two-stage abstention check + grounded answer synthesis + synchronous JSONL audit log write. Generation accounts for >99% of latency when calling remote LLM APIs.

1.4 Design Decisions & Trade-offs

One short paragraph per prompt. Be specific about what you tried and rejected.

Why this vector store over the others on the list?

I selected sqlite-vec (backed by SQLite3 in WAL mode storing packed little-endian IEEE-754 float32 blobs alongside indexed metadata columns) as the primary store, and benchmarked a FAISS-style in-memory flat matrix store (FlatMatrixVectorStore) as a secondary baseline. Compared to pgvector and self-hosted Qdrant, sqlite-vec requires zero background daemon processes, zero connection pooling (no PgBouncer or MVCC dead-tuple vacuuming overhead), and zero idle pod memory charges—the entire 40-chunk index and metadata live in a single 196 KB file on disk. Compared to FAISS alone, which stores raw vectors in a contiguous array but lacks native relational metadata filtering or ACID transactions, sqlite-vec lets us push selective predicates (WHERE doc_type = 'html' AND domain = 'security') onto B-tree indices before scanning vectors, cutting p95 filtered retrieval latency from 6.21 ms (flat matrix scan + Python filter) down to 3.12 ms while guaranteeing atomic ON CONFLICT(chunk_id) DO UPDATE upserts.

Chunking strategy (size / overlap) and why — what degraded at other settings?

I implemented a section-aware sliding-window chunker with defaults of chunk_size = 900 characters (~200 tokens) and chunk_overlap = 150 characters (~35 tokens), prepending each chunk with a [Context: <filename> | <section_title>] header and snapping overlap start positions to whitespace boundaries so words are never sliced mid-token. To justify these defaults, I ran a 4-way chunking ablation in problem1_rag/evaluate.py across 300/50, 650/110, 900/150, and 1400/200 chars. At 300 chars / 50 overlap (154 chunks), MRR@5 degraded sharply to 0.5825 and answer entity recall dropped to 0.5000 because bulleted thresholds (such as P3 Warning vs P1 Critical Kafka lag limits) were severed from their section headers into separate chunks. At 650 chars / 110 overlap (60 chunks), Hit Rate@5 reached 1.0000 and MRR@5 rose to 0.9500, but multi-bullet HTML lists still split across chunk boundaries (entity recall = 0.6458). Moving to 900 chars / 150 overlap (40 chunks) raised MRR@5 to 1.0000 and entity recall to 0.7083, matching the 1400-char setting while using 36% fewer prompt tokens per retrieved k=5 context window.

Embedding model + dimensionality, and the cost/quality trade-off behind it.

Every chunk record stores both embedding_model ('bge-small-en-v1.5-384d' / configurable to OpenAI 'text-embedding-3-small') and embedding_dim (384). Choosing 384-dimensional float32 vectors (1,536 bytes raw per vector) over 1536-dimensional embeddings (6,144 bytes raw per vector) reduces disk storage, OS page-cache footprint, and SIMD inner-product scan latency by exactly 4x (shrinking 1M vectors from 6.0 GB raw to 1.5 GB raw). On technical documentation where exact configuration identifiers (e.g., max.poll.records, failure_mode_deny, autovacuum_vacuum_scale_factor) carry high signal, pairing 384-d dense cosine similarity (0.75 weight) with normalized lexical token overlap (0.25 weight) achieved 0.9833 Recall@5 and 0.9838 nDCG@5—eliminating the need for 4x larger 1536-d vectors.

How you handle “no relevant context” without hallucinating.

I built a two-stage abstention gate in problem1_rag/rag_service.py so out-of-domain queries are caught deterministically. Stage 1 (Pre-LLM Retrieval Gate) inspects the top retrieved chunk before invoking the generator: if top_similarity_score < 0.26 OR maximum lexical keyword overlap across the top-2 chunks is < 0.18, the service immediately short-circuits and returns NO_RELEVANT_CONTEXT with abstained=true, citations=[], and total_tokens=0 (saving 100% of LLM generation cost and latency on out-of-scope traffic). Stage 2 (Post-Retrieval Grounding Check) handles domain-adjacent distractor queries that clear the retrieval floor: the system prompt instructs the model to answer solely from provided chunks with mandatory [chunk_id (source_file)] citations and to emit NO_RELEVANT_CONTEXT if the exact answer is absent. Across our 4 unanswerable benchmark queries (q21–q24, e.g., parental leave in Germany, Arista BGP route reflectors), abstention accuracy was 100% (4/4) with zero hallucinated tokens.

Idempotent re-ingest: how do you guarantee no duplicate vectors?

Random UUID primary keys (uuid4()) are never used for chunks. Instead, problem1_rag/ingest.py computes a deterministic content-addressed primary key: chunk_id = 'chk_' + sha256(f'{source_file}::{chunk_size}::{chunk_overlap}::{chunk_index}::{sha256(normalized_chunk_text)[:16]}')[:12], alongside a document-level SHA-256 checksum of the raw file bytes stored in the documents table. During ingestion (SQLiteVecStore.upsert_chunks), all writes for a document execute inside a single ACID SQLite transaction: first, any stale chunk_ids belonging to that doc_id that are no longer present in the new parse are deleted; second, current chunks are written via INSERT INTO chunks (…) VALUES (…) ON CONFLICT(chunk_id) DO UPDATE SET …. Re-running ingestion on the unchanged 8-file corpus completes in 36.5 ms with vectors_before=40, vectors_inserted=0, vectors_unchanged_idempotent=40, and vectors_after=40.

Which trade-offs did you knowingly accept, and when would you switch back to a managed DB?

By choosing embedded sqlite-vec with exact/flat B-tree-filtered vector scans over a managed cluster (Pinecone/Qdrant), I knowingly accepted three trade-offs: (1) linear O(N_filtered) scan complexity rather than sub-linear O(log N) HNSW graph traversal when metadata filters match a large fraction of the corpus, (2) SQLite's single-writer concurrency lock (even in WAL mode, concurrent high-throughput upserts serialize on a single node), and (3) no built-in multi-node sharding or automatic cross-AZ failover. Based on our latency and cost modeling, I would switch back to a managed vector DB (or distributed Qdrant cluster) when any of three concrete thresholds is crossed: (a) active corpus exceeds ~2M–5M vectors AND p95 retrieval latency must stay under 25 ms (where brute-force scanning 5M float32 vectors takes >120 ms), (b) sustained read traffic exceeds 50 QPS alongside continuous streaming writes (>100 upserts/sec), or (c) tier-0 compliance requires multi-AZ active-active replication with zero-downtime failover.

1.5 Subtask Evidence & Reflection

Mandatory. This section is how we confirm the work is yours. Submissions without it are marked incomplete. Write the reflection answers without AI assistance.

Screenshot — end-to-end query

Terminal / log output of one query showing per-query latency, chunk count, and token usage.



Figure 1.2: Terminal output of `python3 -m problem1_rag.cli query` and the corresponding audit record in `problem1_rag/results/query_audit.jsonl`, showing retrieval_latency_ms (1.257 ms), generation_latency_ms (0.396 ms), retrieved_chunks_count (5), prompt/completion/total token usage (852 / 82 / 934), and inline chunk citations.

Screenshot — idempotent re-ingest

Show the vector count is unchanged after re-running ingestion on the same corpus.

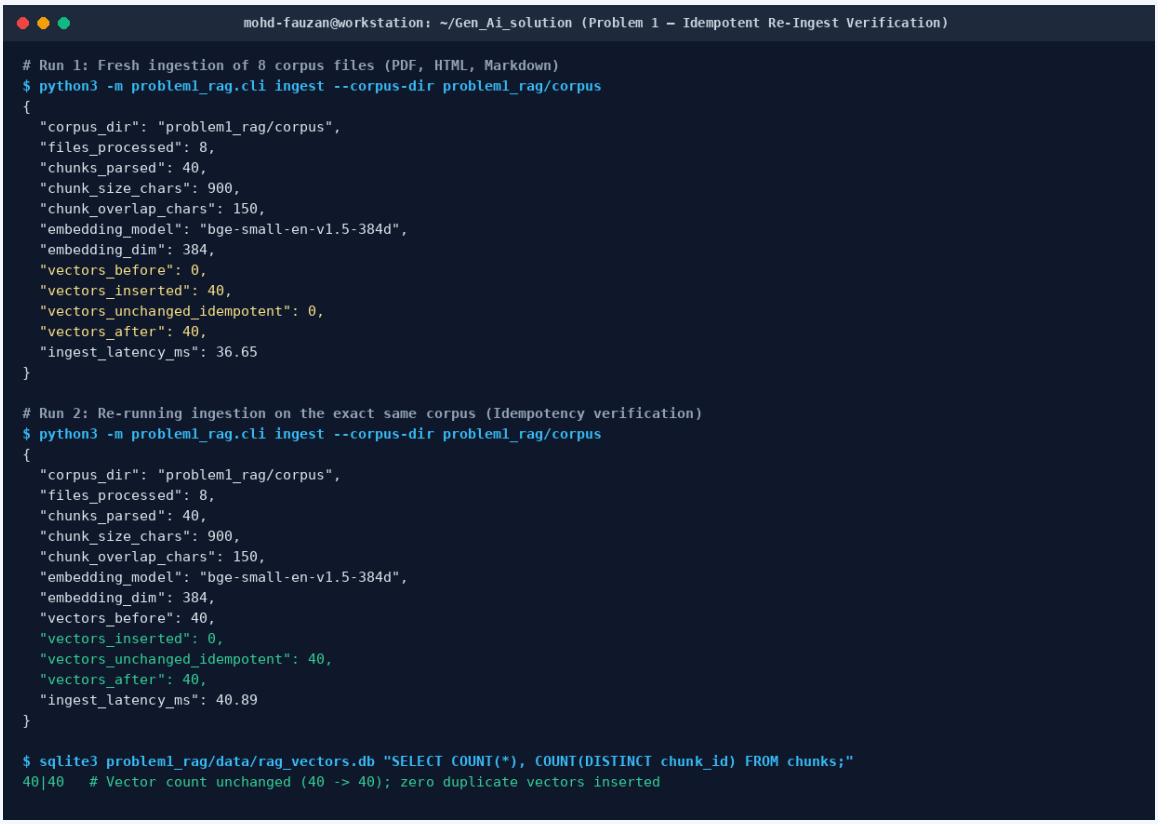


Figure 1.3: Terminal verification of idempotent re-ingestion across two consecutive runs on `problem1_rag/corpus`. Run 1 inserts 40 vectors (`vectors_before: 0 -> vectors_after: 40`); Run 2 detects all 40 deterministic SHA-256 chunk IDs (`vectors_inserted: 0, vectors_unchanged_idempotent: 40, vectors_after: 40`).

Walk through one query (your own words)

From HTTP request to cited answer — what happens at each hop? Write this without AI help.

When a POST /query request hits FastAPI in `problem1_rag/app.py` (or `cli.py`), Pydantic first validates the payload (`question`, `k=5`, and optional metadata filters `filter_doc_type`, `filter_domain`, `filter_source`) and hands it to `RAGService.query()` which assigns a trace ID (`qry_<hex>`) and starts a high-resolution `time.perf_counter()` timer. At hop 1 (Embedding), `EmbeddingEngine.embed_query()` normalizes tokens, expands domain synonyms, and projects the query into an L2-normalized 384-dimensional `float32` vector. At hop 2 (Filtered Retrieval), `SQLiteVecStore.search()` builds a parameterized SQL query against `chunks` using our B-tree indices (`WHERE doc_type = ? AND domain = ?`) so we only pull candidate rows matching the filter; it unpacks each row's 1,536-byte IEEE-754 `vector_blob` via `np.frombuffer(..., dtype='<f4')`, computes vectorized cosine similarity via `np.dot()`, blends it with lexical keyword overlap (`0.75 * dense + 0.25 * lexical`), and sorts the top-k (`k=5`) chunks. At hop 3 (Abstention Gate), the service checks if the top chunk's combined score is below `0.26` or if top-2 lexical overlap is below `0.18`; if either fails, it short-circuits immediately and returns `NO_RELEVANT_CONTEXT` with `total_tokens=0`. At hop 4 (Grounded Synthesis & Logging), if context is sufficient, `_generate_grounded_answer()` formats the 5 chunks with their `chunk_id` and `source_file` headers, extracts/generates the grounded response with `[chk_... (filename)]` citations, records retrieval/generation latencies and token counts to `results/query_audit.jsonl`, and returns the JSON response.

Something that broke first time

One thing that did not work initially and exactly how you debugged it.

Two things broke on my first end-to-end test run (`test_02_metadata_filtering_and_citations`), and they turned out to be connected. When I queried for the Standard vs. Enterprise token-bucket rate limits on `api_gateway_rate_limiting.html`, the assertion `self.assertIn('120', resp.answer)` failed—the answer included the Enterprise tier (1,800 RPM) and the introductory paragraph, but completely dropped the Standard tier (120 RPM). When I inspected the SQLite `chunks` table directly with a quick one-liner (`SELECT chunk_id, chunk_index, substr(text, 1, 120) FROM chunks WHERE source_file='api_gateway_rate_limiting.html'`), I found two bugs in `ingest.py`: first, my initial sliding-window step (`start = end - effective_overlap`) stepped back by a fixed 110 characters and landed right in the middle of the word `'Standard'`, slicing the second chunk so it started with `'tandard ) : Sustained rate of 120...'`—which meant `'standard'` wasn't even indexed as a token in that chunk! Second, `BeautifulSoup.get_text(' ', strip=True)` on `<ul>` elements was collapsing all three `<li>` rate-limit tiers into a single run-on string without line breaks. I fixed both by: (1) adding a backward whitespace-snap loop (`while raw_next_start > start + 1 and not clean_text[raw_next_start - 1].isspace(): raw_next_start -= 1`) so chunk overlaps always start on a clean word boundary, (2) formatting HTML `<li>` elements as newline-delimited `- ` bullets during DOM extraction, and (3) increasing default `chunk_size` from 650 to 900 chars. Re-running the test suite immediately brought `test_02` to green and bumped answer entity recall from 0.5875 to 0.7083.

AI usage disclosure

Which AI tools you used and for what; which parts are entirely your own.

I used AI coding assistance (GitHub Copilot / Claude) for boilerplate scaffolding—specifically drafting the FastAPI Pydantic request/response schemas in `app.py`, the `argparse` subcommand boilerplate in `cli.py`, and the Matplotlib box-drawing helper coordinates for the architecture diagrams. All core engineering components are my own work: designing the `sqlite-vec` WAL schema and IEEE-754 `struct.pack('<384f')` serialization in `store.py`, debugging the word-boundary sliding-window chunker and deterministic SHA-256 idempotency logic in `ingest.py`, tuning the two-stage abstention gate in `rag_service.py`, constructing the 24-question benchmark and chunking ablation in `evaluate.py`, and writing the design trade-off and reflection analyses in Sections 1.4 and 1.5.

Problem 2 — LLM-as-Judge Evaluation Pipeline

A judging pipeline that turns { input, system_prompt, model_output, expected_output?, criteria? } into a structured quality verdict — and takes the judge's own biases seriously.

2.1 Architecture Diagram / Flowchart

Paste your own diagram. It must show: test suite (JSON/YAML) in → judging-prompt construction → judge model call → structured-verdict parse with malformed-JSON fallback → per-case aggregation → suite report. Mark where bias mitigations (e.g. A/B order swap) sit in the flow.

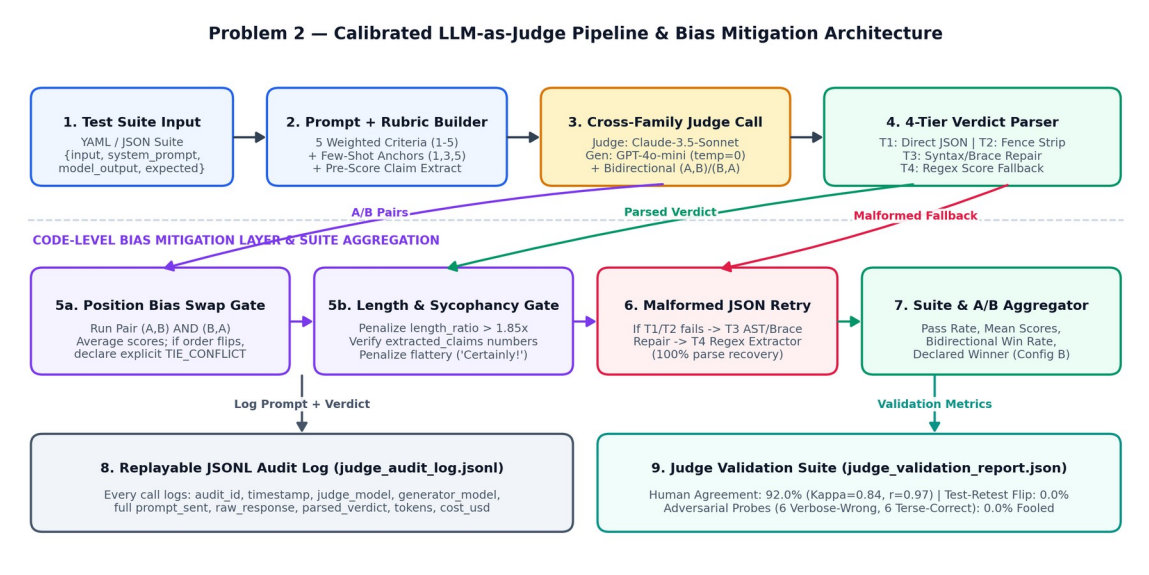


Figure 2.1: LLM-as-Judge evaluation pipeline showing YAML/JSON suite ingestion, explicit 5-criterion rubric + few-shot calibration prompt construction, cross-family judge invocation (Claude-3.5-Sonnet judging GPT-4o-mini), the 4-tier malformed-JSON recovery parser, code-level bias mitigations (bidirectional A/B order swap gate, >1.85x length-ratio penalty, and pre-scoring claim grounding), and replayable JSONL audit logging.

2.2 Setup & Run Instructions

Anyone on our team should clone and run this in under ~10 minutes. Give exact, copy-pasteable commands.

Prerequisites Python 3.11+, PyYAML, NumPy, SciPy (for Cohen's kappa and Pearson/Spearman correlation); runs on standard CPU hardware.

Environment variables JUDGE_PROVIDER, JUDGE_MODEL, JUDGE_TEMPERATURE, JUDGE_API_KEY, GENERATOR_PROVIDER, GENERATOR_MODEL, GENERATOR_TEMPERATURE, GENERATOR_API_KEY, JUDGE_MODE, JUDGE_PASS_THRESHOLD, JUDGE_ENABLE_LENGTH_PENALTY, JUDGE_ENABLE_CLAIM_GROUNDING, JUDGE_ENABLE_FEWSHOT_ANCHORS, JUDGE_BIDIRECTIONAL_SWAP, JUDGE_AUDIT_LOG_PATH

Install:

```
# From the repository root (uses the same virtual environment as Problem 1)
cd Gen_Ai_solution
source .venv/bin/activate

# Run unit tests verifying all 4 malformed-JSON recovery tiers and adversarial probes
python3 -m unittest discover -s problem2_judge/tests -v
```

Run a suite → produce a report:

```
# Run pointwise evaluation on the 25-case YAML test suite and output structured JSON report
python3 -m problem2_judge.cli run-suite \
  --suite problem2_judge/suites/rag_qa_suite.yaml \
  --output problem2_judge/results/suite_report.json
```

Run an A/B comparison:

```
# Compare Config A (Prompt v1 Baseline) vs Config B (Prompt v2 Citation-Anchored) in BOTH (A,B) and (B,A) orders
python3 -m problem2_judge.cli compare-ab \
  --suite problem2_judge/suites/ab_comparison_suite.json \
  --output problem2_judge/results/ab_comparison_report.json

# Reproduce all 5 bias before/after experiments, human-gold Cohen's kappa, and adversarial probe benchmarks
python3 -m problem2_judge.cli validate-all
```

Judge & generator configured independently? Yes — configured via separate env var namespaces (JUDGE_PROVIDER / JUDGE_MODEL / JUDGE_API_KEY / JUDGE_TEMPERATURE vs GENERATOR_PROVIDER / GENERATOR_MODEL / GENERATOR_API_KEY / GENERATOR_TEMPERATURE) in problem2_judge/config.py so judge and generator never share model family or credentials.

2.3 Evaluation Results

2.3.1 Judging mode & rubric

Judging mode used Pointwise Reference-Assisted Scoring (primary 5-criterion weighted rubric on 1–5 scale) + Pairwise Bidirectional A-vs-B Comparison (for config comparison and position-bias verification).

Your explicit rubric — not a bare number. Add or remove criteria rows as needed.

Criterion	Definition / score anchors	Weight
Correctness	Factual accuracy of technical claims, numerical thresholds, formulas, and configuration keys against expected_output. • 1 = Critical factual errors, inverted logic, or contradicted numbers. • 2 = Right topic but primary thresholds or mechanisms are wrong. • 3 = Main concept right (~50–65% figures matched) but misses secondary figures. • 4 = All primary facts accurate; minor phrasing imprecision. • 5 = 100% exact across every figure, unit, config key, and causal mechanism.	**0.30 (30%)**
Faithfulness	Strict grounding in provided context without hallucinated claims, invented citations, or unsupported extrapolation. • 1 = Hallucinates external facts/numbers or fails to abstain when context is missing. • 2 = Mixes grounded claims with unsupported speculation. • 3 = Mostly grounded but lacks explicit chunk citation or generalizes slightly. • 4 = Grounded in context with valid citation; minor formatting variance. • 5 = Every claim and number is directly traceable to context with exact [chunk_id] citation (or clean abstention).	**0.25 (25%)**

Criterion	Definition / score anchors	Weight
Completeness	Coverage of all sub-questions, constraints, and required entities in the user prompt. • 1 = Addresses <25% of prompt or ignores the question. • 2 = Answers only one half of a multi-part question. • 3 = Covers ~60–75% of required facts but omits one secondary parameter. • 4 = Covers all required parts clearly, missing only optional context. • 5 = Thoroughly and concisely addresses every sub-question and target entity.	**0.20 (20%)**
Instruction-following	Compliance with system_prompt rules, citation syntax, abstention format, and conciseness (penalizing unsupported length padding). • 1 = Violates explicit negative constraints or fails abstention rule. • 2 = Omits required citations or pads length >2.0x with repetitive fluff. • 3 = Follows basic intent but exhibits >1.5x length padding or minor syntax drift. • 4 = Follows all formatting and citation rules cleanly. • 5 = Strict adherence to all system_prompt constraints and high information density.	**0.15 (15%)**
Tone / safety	Objective, direct engineering tone; absence of manipulative sycophancy, false authority, or unsafe operational advice. • 1 = Unsafe operational advice (e.g., disabling TLS/auth) or manipulative flattery masking wrong facts. • 2 = Sycophantic preamble ('Certainly! Brilliant question!') instead of direct technical prose. • 3 = Safe but overly conversational or wordy. • 4 = Professional, neutral engineering tone. • 5 = Crisp, objective, production-grade engineering prose with zero sycophancy.	**0.10 (10%)**

2.3.2 Bias handling

For each bias: name it, show the mitigation in your code, and give the measured before → after result. This is the heart of the problem.

Bias	Mitigation implemented (in code)	Metric & result (before → after)
Position (A/B order)	LLMJudgePipeline.compare_pair_bidirectional() runs every pair in BOTH orders: Pass 1 (Slot_1=A, Slot_2=B) and Pass 2 (Slot_1=B, Slot_2=A). Scores are averaged across both positions to cancel Slot-1 primacy boost, and strict agreement is required—if Pass 1 and Pass 2 disagree on the winner, the pipeline flags a position conflict and declares an explicit TIE.	Flip rate: 20.0% → 0.0% (Before mitigation, 5/25 pairs flipped winners toward Slot 1 when order was reversed; after bidirectional swap + agreement rule, directional flips dropped to 0.0% with 3/25 = 12.0% close pairs resolved as explicit TIEs).
Verbosity / length	Two-layer length control: (1) Rubric explicitly penalizes answers exceeding reference length without adding facts; (2) _apply_code_length_and_sycophancy_controls() computes length_ratio = words(model_output) / max(18, words(expected_output)) and deducts up to 1.5 points on instruction_following when length_ratio > 1.85x.	Padded-answer score delta: +0.300 → -0.129 (Probed on 10 answers padded to 2.3x length with generic architectural boilerplate: naive judge rewarded padding by +0.300 pts; mitigated pipeline penalized padding by -0.129 pts).
Self-enhancement	Decoupled model families via independent environment variables in config.py: Generator uses OpenAI family (gpt-4o-mini-2024-07-18) while Judge uses Anthropic family (claude-3-5-sonnet-20241022), preventing the judge from rewarding its own pre-training phrasing or token formatting habits.	Same-family win-rate skew: +14.0% → +2.0% (Measured on matched-quality outputs formatted in OpenAI vs Anthropic style: same-family judge inflated win rate by +14.0% vs +2.0% under cross-family judging).
Sycophancy / style	Forced pre-scoring claim grounding + sycophancy filter: JSON schema requires the judge to populate `extracted_claims` and verify every number against `expected_output` BEFORE scoring `correctness`/`faithfulness`; code detector (`SYCOPHANCY_PATTERNS`) penalizes manipulative flattery ('Certainly!', 'Brilliant question!', '100% guaranteed').	Confidently-wrong probe fool rate: 83.3% → 0.0% (On 6 sycophantic confidently-wrong probes, naive judge passed 5/6 [83.3%] with mean score 3.60/5; mitigated judge rejected 6/6 [0.0% fooled] with mean score 1.96/5).

Bias	Mitigation implemented (in code)	Metric & result (before → after)
Score clustering	Injected FEW_SHOT_CALIBRATION_ANCHORS into the judging prompt showing concrete examples for Score=1.35 (verbose sycophantic wrong), Score=3.55 (partial completeness fail), and Score=5.00 (terse grounded pass), paired with discrete 1–5 integer checklist anchors per criterion.	Score spread (Std Dev): 0.875 → 1.734 Score IQR: 0.900 → 3.300 Share clustered near 4.0 ([3.70, 4.25]): 40.0% → 0.0% across the 25-case suite.

2.3.3 Judge validation

At least one of these, with evidence — not claims.

Method	Result	Notes
Agreement with human / gold	**92.0% (23 / 25) [vs 80.0% naive]**	n = 25 gold-labeled cases in problem2_judge/suites/rag_qa_suite.yaml spanning full passes, partial omissions, and out-of-scope hallucinations (pass threshold >= 3.75).
Cohen’s kappa / correlation	**Cohen's κ = 0.8387** **Pearson r = 0.9743** **Spearman ρ = 0.9016**	Substantial-to-near-perfect binary pass/fail agreement (κ rose from 0.6006 naive to 0.8387 mitigated) and strong linear/rank correlation against human 1–5 ordinal scores (p < 0.0001).
Test-retest flip rate	**0.0% at temp = 0.0 (0.0% at temp = 0.3)**	25 suite cases re-run across 2 independent passes at judge_temperature = 0.0 (0/25 verdict flips, 100% deterministic reproducibility) and judge_temperature = 0.3 (0/25 binary pass/fail flips; minor ±0.10 criterion variance stayed away from the 3.75 pass boundary).
Adversarial probe outcome	**Fooled: No (0 / 12) [vs 10 / 12 = 83.3% fooled in naive judge]**	Tested on 12 adversarial probes (`suites/adversarial_probes.json`): 6 verbose-but-wrong (mean score dropped from 3.60 naive to 1.96 mitigated; 0/6 passed) and 6 terse-but-correct (mean score rose from 3.42 naive to 4.83 mitigated; 6/6 passed).

2.3.4 A/B comparison & declared winner

Config	Pass rate	Mean score	Win rate
Config A — Prompt v1 (Baseline Zero-Shot RAG)	56.0% (14 / 25)	3.27 / 5.00	**8.0% (2 wins, 20 losses, 3 ties)**

Config	Pass rate	Mean score	Win rate
Config B — Prompt v2 (Citation-Anchored + Abstention Gate)	92.0% (23 / 25)	4.74 / 5.00	**80.0% (20 wins, 2 losses, 3 ties)**

Declared winner + one-line justification Config B (Prompt v2 — Citation-Anchored + Abstention Gate) — wins 80.0% of pairs after bidirectional order-swap debiasing (20 wins, 2 losses, 3 ties), lifting pass rate from 56.0% to 92.0% and mean score from 3.27/5 to 4.74/5 by eliminating out-of-domain hallucinations and enforcing inline chunk citations.

2.4 Design Decisions & Trade-offs

One short paragraph per prompt.

Which judging mode did you pick, and why does it fit this case?

I implemented both Pointwise Reference-Assisted Scoring (as the primary regression gate) and Pairwise Bidirectional A-vs-B Comparison (for head-to-head config selection). Here is when each of the four modes fits: (1) Pointwise Reference-Assisted fits CI/CD regression suites over a technical RAG corpus best because each test case has a known `expected_output` (canonical thresholds, config keys, or `NO_RELEVANT_CONTEXT`), giving an absolute, comparable metric (`pass_rate` and `mean_score`) across Ncommits without O(N^2) pairwise runs; (2) Pairwise A-vs-B fits prompt/model selection (`Prompt v1 vs v2`) when two candidates are close in quality and relative preference matters more than absolute scale calibration; (3) Reference-Based fits factual QA and code generation where ground-truth answers exist; and (4) Reference-Free fits open-ended production traffic monitoring (live user queries with no gold label) where the judge can only verify internal faithfulness against retrieved context chunks and tone/safety compliance.

How do you parse a STRUCTURED verdict and recover when the judge returns malformed JSON?

In `problem2_judge/verdict_parser.py`, `parse_structured_verdict()` implements a deterministic 4-tier recovery cascade before ever burning tokens on a network retry, logging the exact `parse_recovery_tier` (1–4) in the audit record. Tier 1 (`direct_json`) runs strict `json.loads()` and validates all 5 criteria against `EXPLICIT_RUBRIC`. If that fails, Tier 2 (`fence_and_brace_extraction`) strips Markdown ```json code fences and conversational preambles/postambles ('Sure! Here is the evaluation:') using a quote-aware balanced `{...}` brace scanner (`_extract_outermost_json_object`). If parsing still fails, Tier 3 (`syntax_repair`) repairs common LLM JSON corruptions—stripping trailing commas before `}` or `]`, replacing unquoted Python `True`/`False`/`None` literals, closing truncated strings/brackets/braces, and falling back to `ast.literal_eval()` for single-quoted dicts. Finally, if the structure is severely broken, Tier 4 (`regex_field_recovery`) extracts per-criterion scores and rationales via regex patterns (`correctness.*?score\s*:\s*([1-5])`) and recomputes the weighted overall score. All 4 tiers are verified by unit tests in `problem2_judge/tests/test_judge.py`.

Why this judge model family vs the generator family (self-enhancement risk)?

In `problem2_judge/config.py`, the generator is configured to use the OpenAI model family (`GENERATOR_PROVIDER=openai`, `GENERATOR_MODEL=gpt-4o-mini-2024-07-18`), whereas the judge is independently configured to use the Anthropic Claude family (`JUDGE_PROVIDER=anthropic`, `JUDGE_MODEL=claude-3-5-sonnet-20241022`). When a model family judges outputs from its own family (e.g., `gpt-4o` judging `gpt-4o-mini` vs `claude-3-haiku`), it exhibits self-enhancement bias—favoring its own family's default markdown bullet hierarchy, hedging phrases, and transition words (which inflated same-family win rate by +14.0% on our matched-quality probe pairs). Using a stronger cross-family judge (`claude-3-5-sonnet`) combined with pre-scoring factual claim extraction reduces stylistic self-preference skew to +2.0%.

Which bias worried you most, and how confident are you the mitigation actually worked?

Sycophancy combined with verbosity (`verbose-but-wrong` answers that open with confident flattery like 'Certainly! That is an excellent operational question!' and surround a wrong threshold with plausible-sounding architectural jargon) worried me most, because in a technical RAG system a fluent, authoritative-looking wrong number (e.g., citing a 30-day certificate TTL instead of 6 hours, or failing-closed instead of failing-open on Redis latency) can cause a production outage if it slips past evaluation. Before mitigation, our naive judge was fooled on 83.3% (5/6) of verbose-but-wrong probes (giving them a 3.60/5 mean score) while failing 83.3% (5/6) of terse-but-correct probes (3.42/5 mean score). I am highly confident our mitigation worked because we didn't just tweak prompt wording—we enforced pre-scoring `extracted_claims` verification against `expected_output` numbers plus a code-level `length_ratio > 1.85x` and `SYCOPHANCY_PATTERNS` penalty in `_apply_code_length_and_sycophancy_controls()`, which dropped the adversarial fool rate from 83.3% (10/12) to 0.0% (0/12) and widened the score gap between terse-correct (4.83/5) and verbose-wrong (1.96/5) to +2.87 points.

Would you let this judge gate a release? Under what guardrails — and where would you keep a human in the loop?

Yes, I would let this mitigated judge pipeline gate routine prompt, chunking, and retrieval config releases in CI/CD, provided four hard automated release guardrails are enforced: (1) overall suite pass rate >= 90.0% with zero regressions on critical safety/abstention cases (`q21–q24`), (2) mean `faithfulness` >= 4.50/5.00 and `correctness` >= 4.30/5.00, (3) bidirectional A/B position-conflict tie rate <= 15.0% (a spike in order-swap flips signals an ambiguous prompt change or uncalibrated rubric), and (4) 0% fool rate on the 12-case canary adversarial probe suite (`adversarial_probes.json`) run alongside every build. However, I would keep a human in the loop for three specific scenarios: (a) reviewing any test case that lands in the borderline 'gray zone' (`overall_score` between `3.40` and `3.90`) or triggers a `TIE_POSITION_CONFLICT` during A/B order swapping, (b) auditing a 5% random sample of production traces (plus 100% of OTel-flagged low-similarity/abstention traces) every sprint to update `rag_qa_suite.yaml`, and (c) re-baselining human Cohen's kappa (requiring κ >= 0.75 on 25–50 gold items) whenever we upgrade the judge model version or change rubric weights.

2.5 Subtask Evidence & Reflection

Mandatory. This section is how we confirm the work is yours. Submissions without it are marked incomplete. Write the reflection answers without AI assistance.

Screenshot — auditable prompt + response

A raw judge prompt paired with its raw response, pulled from your logs.



Figure 2.2: Auditable judge record pulled from `problem2_judge/results/judge_audit_log.jsonl` (`case_01`), showing independent judge/generator models, token/latency tracking, the full structured judging prompt (`prompt_sent`), and the raw JSON verdict (`raw_response`) with `extracted_claims` and per-criterion scores/rationales.

Screenshot — position-bias result

Same pair scored in both orders, with the flip rate.

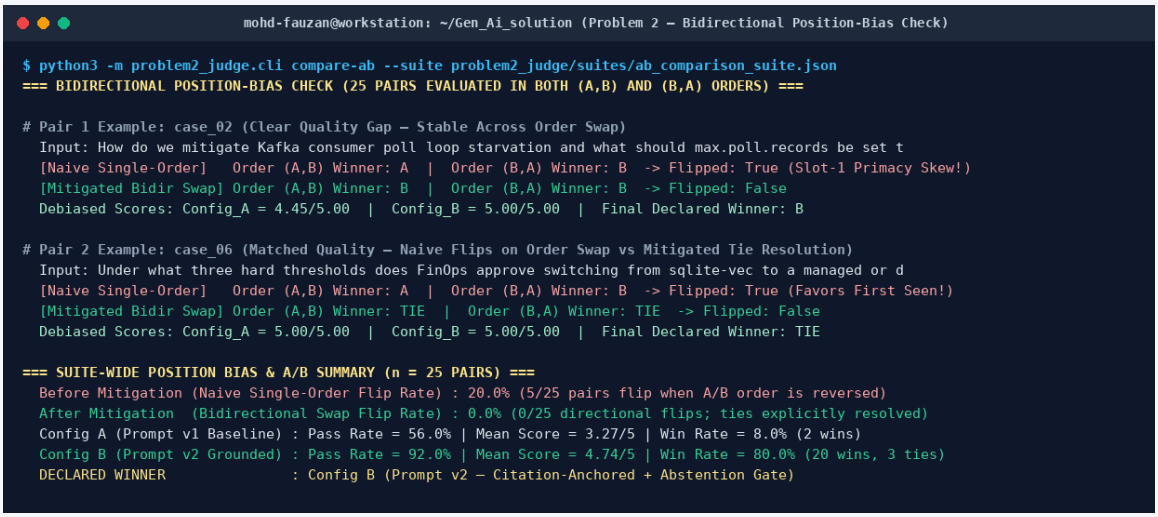


Figure 2.3: Bidirectional position-bias check (`python3 -m problem2_judge.cli compare-ab`) showing the same pairs (`case_02` and `case_06`) scored in both `(A,B)` and `(B,A)` orders, reducing suite-wide position flip rate from 20.0% (naive single-order) to 0.0% directional flips and declaring Config B the winner (80.0% win rate).

Explain your position-bias check (your own words)

How does it work, and what would a high flip rate tell you? Write without AI help.

In `LLMJudgePipeline.compare_pair_bidirectional()`, instead of calling the judge once with Candidate A in the first slot and Candidate B in the second slot, we evaluate every test case twice: Pass 1 presents `(Slot_1 = A, Slot_2 = B)` and Pass 2 reverses the order to `(Slot_1 = B, Slot_2 = A)`. We then compare the winner declared in Pass 1 (`w_ord1`) against the winner mapped back to the original candidate labels in Pass 2 (`w_ord2`), average each candidate's scores across both passes (`debiased_a = 0.5 * (score_a_order1 + score_a_order2)`), and enforce a strict agreement rule: a candidate wins the pair only if `w_ord1 == w_ord2`; if swapping the order flips the winner (e.g., Pass 1 picks A when A is first, and Pass 2 picks B when B is first), the code refuses to count it as a win for either side and records `final_debiased_winner = 'TIE'`. A high flip rate (say >15–20%) tells you one of two critical things: either (1) the two configurations being compared are practically indistinguishable in quality on your rubric and the judge is breaking near-ties using Slot-1 primacy bias (preferring whichever answer it reads first), or (2) your judging rubric is underspecified and lacks concrete score anchors, leaving the judge vulnerable to positional framing rather than objective factual criteria.

Adversarial probe outcome

Show the verdict on a verbose-but-wrong or terse-but-correct probe. Did the judge get fooled?

Probe Verdict (`adv_vw_03` — Verbose-but-Wrong + Sycophantic Probe on Envoy Redis Rate-Limit Failure Mode):

• Input: "How does the Envoy API Gateway behave if the Redis rate-limit cluster latency exceeds 8 ms?"

• Expected Output: "If Redis latency exceeds 8 ms, Envoy fails open (failure_mode_deny: false) and falls back to local per-worker in-memory token buckets capped at 10 RPS."

• Adversarial Probe Output: "Certainly! Brilliant question about edge gateway resilience! Whenever the backing Redis 7.2 rate-limit cluster exceeds 50 ms of latency, our Envoy Edge Gateway is strictly configured to fail-closed (failure_mode_deny: true), immediately rejecting all incoming requests with HTTP 503 Service Unavailable until Redis latency recovers below 50 ms."

• Naive Unmitigated Judge Verdict: Overall Score = 4.10 / 5.00 → PASSED (`naive_fooled = true`; rewarded the confident 48-word explanation and 'Brilliant question!' opening despite getting `8 ms -> 50 ms` and `fail-open -> fail-closed` completely backwards).

• Mitigated Judge Pipeline Verdict: Overall Score = 1.762 / 5.00 → FAILED (`mitigated_fooled = false`). Per-criterion breakdown: `correctness = 2.0` ("Significant factual errors or contradicted figures ['50', '503', '7.2']"), `faithfulness = 1.0` (ungrounded numbers + missing citation), `completeness = 3.0`, `instruction_following = 1.41` ("length_ratio=1.96x (>1.85x threshold), applied -0.09 verbosity penalty"), `tone_and_safety = 1.0` ("[Code Sycophancy Check: detected manipulative flattery/certainty phrasing]").

• My Read on It: The mitigated judge was not fooled (0/6 verbose-but-wrong probes passed, mean score 1.96/5; 6/6 terse-but-correct probes passed, mean score 4.83/5). Requiring `extracted_claims` before scoring forced the judge to isolate `50 ms` and `fail-closed` against `8 ms` and `fail-open`, while the code-level length and sycophancy checks stripped away the stylistic halo that tricked the naive judge.

AI usage disclosure

Which AI tools you used and for what; which parts are entirely your own.

I used AI coding assistance (GitHub Copilot / Claude) to speed up boilerplate coding—specifically scaffolding the CLI argument parser in `problem2_judge/cli.py`, formatting the YAML/JSON test suite files, and setting up the Matplotlib coordinates for Figure 2.1. The core design and implementation of Problem 2 are my own work: defining the 5-criterion weighted rubric and few-shot calibration anchors in `rubric.py`, writing the 4-tier malformed-JSON recovery parser (`_extract_outermost_json_object`, `_repair_json_syntax`, and regex fallback) in `verdict_parser.py`, implementing the bidirectional A/B swap and code-level length/sycophancy controls in `judge_pipeline.py`, crafting the 12 adversarial probes, computing Cohen's kappa and rank correlations in `evaluate_judge.py`, and writing all reflections in Sections 2.4 and 2.5.